ESPECIFICACIÓN DE REQUISITOS DEL SOFTWARE (ERS)

Audi F1 Store — Versión 2.0 (Evaluación Parcial 2)

DSY1104 Desarrollo Fullstack II · 17 de octubre de 2026 · Repositorio: github.com/McRodriFly/Fullstack-02

1. Descripción general del proyecto

La Audi F1 Store es una tienda online de merchandising (ficticia) del equipo Audi Formula 1. Esta versión 2.0 del ERS corresponde a la Evaluación Parcial 2: la tienda construida en la EP1 con HTML, CSS, JavaScript y Bootstrap se migró a **React**, con componentes reutilizables, navegación con React Router, formularios validados, persistencia simulada en `localStorage` y un proceso de pruebas unitarias con **Jasmine y Karma**.

2. Objetivos de la actualización (EP2)

- Migrar el frontend a React 19 con componentes que reciben datos por **props** y manejan su **estado** con hooks.
- Implementar navegación SPA con React Router 7: rutas, parámetros (`:id`), parámetros de búsqueda (`?cat=`) y rutas anidadas para el panel de administración.
- Mantener un diseño responsivo con la grilla y los componentes de **Bootstrap 5** para celular, tablet y escritorio.
- Simular una base de datos con un archivo JavaScript (CRUD) y persistencia en `localStorage`.
- Agregar vistas de negocio: categorías, ofertas, checkout, compra exitosa y fallida, y panel de administración.
- Configurar un entorno de pruebas con Karma, Jasmine, React Testing Library y medición de cobertura.

3. Propuesta personalizada

Respecto de la plantilla base, el equipo agregó:

- **Cupones de descuento** (`F1AUDI` 10 %, `NEUBURG2026` 15 %) aplicados en el carrito y respetados en el checkout.
- **Checkout con simulación de pago**: despacho o retiro en tienda, validación de tarjeta y una tarjeta de prueba (`4000000000000002`) que simula un rechazo sin perder el carrito.
- **Validación de RUT chileno** (módulo 11) y selects encadenados región/comuna en el registro.
- **Panel de administración** con KPIs calculados en vivo, alerta de stock crítico (menos de 5 unidades) y mantenedores de productos y usuarios.
- **Notificación (toast)** al agregar productos, que reemplaza los `alert()` de la EP1.

4. Arquitectura y tecnologías

Tecnología	Uso en el proyecto
React 19 + Vite	Biblioteca de componentes y herramienta de compilación y servidor de desarrollo.
React Router 7	Navegación SPA: `Routes`, `NavLink`, `useParams`, `useSearchParams`, `useLocation`, `useNavigate` y `Outlet`.
Bootstrap 5 + Bootstrap Icons	Grilla responsiva, tarjetas, formularios, tablas, alertas e íconos.
Context API	`CarritoContext` comparte el carrito entre el Navbar, las vistas y el toast.
Jasmine + Karma + Testing Library	Pruebas unitarias en Chrome Headless con cobertura (karma-coverage / Istanbul).

Estructura de carpetas: `components/` (piezas reutilizables), `pages/` y `pages/admin/` (vistas), `context/` (estado global), `data/` (base de datos simulada), `utils/` (validadores puros), `styles/` y `tests/`.

Componentes principales: props y estado

Componente	Props	Estado / hooks
ProductCard	`producto`, `onAgregar`	Sin estado propio. Renderizado condicional del precio de oferta.
Navbar	—	`menuAbierto` (menú móvil) + `cantidad` desde `useCarrito()`.
CarritoProvider	`children`	`carrito`, `descuento`, `toast`; `useEffect` oculta el toast a los 2,5 s.
ErrorBoundary	`children`	Componente de clase con estado `hayError`.
Login, Registro, Contacto, Checkout	—	Formularios controlados (`useState`) y mensajes de error.
Categorías	—	Filtro guardado en la URL con `useSearchParams`.
DetalleProducto	—	Lee el `:id` con `useParams`.
AdminLayout / AdminProductos	—	`menuAbierto`; `form`, `idEditando` y `productos` para el CRUD.

5. Requisitos funcionales

ID	Requisito	Implementación
RF01	Catálogo y detalle de producto	Home con `ProductCard` y vista `/producto/:id`; aviso si el producto no existe.
RF02	Categorías y ofertas	`/categorias` con filtro por botón y por URL; `/ofertas` con productos rebajados.
RF03	Carrito y cupones	Agregar, eliminar, subtotal, descuento y total; persistencia en `localStorage`.
RF04	Checkout y resultado del pago	Formulario validado por campo; vistas `/compra-exitosa` y `/compra-fallida`.
RF05	Login y registro	Dominios permitidos, contraseña de 4 a 10 caracteres, RUT módulo 11; un correo con "admin" redirige a `/admin`.
RF06	Contacto	Formulario validado con confirmación en pantalla.
RF07	Panel de administración	Dashboard de KPIs, alerta de stock crítico, CRUD de productos y alta/baja de usuarios.
RF08	Manejo de errores	Página 404 para rutas inexistentes y `ErrorBoundary` ante errores de renderizado.

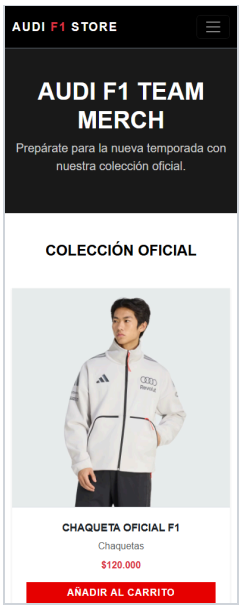
6. Requisitos no funcionales

- **RNF01 – Rendimiento:** SPA sin recargas de página. El build de producción con Vite genera un JavaScript de 313 kB (95 kB comprimido con gzip).
- **RNF02 – Responsividad:** la interfaz se adapta a celular, tablet y escritorio (revisado a 390, 820 y 1400 px) usando la grilla de Bootstrap (ver sección 7).
- **RNF03 – Mantenibilidad:** componentes con una sola responsabilidad, validaciones como funciones puras en `utils/validadores.js` y revisión estática con oxlint sin advertencias.
- **RNF04 – Calidad y testeabilidad:** 94 pruebas unitarias automatizadas con Jasmine y Karma; cobertura mínima exigida de 90 % en sentencias, ramas, funciones y líneas (configurada en Karma), alcanzada en 100 %.
- **RNF05 – Seguridad básica:** React muestra los datos ingresados como texto (no interpreta HTML) y el proyecto no usa `dangerouslySetInnerHTML`.

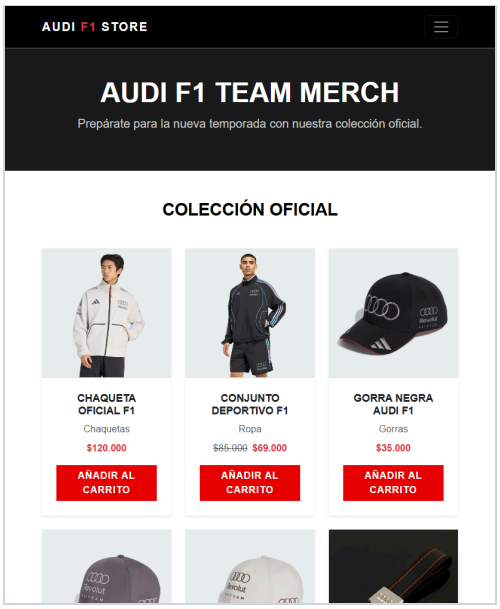
7. Diseño responsivo con Bootstrap

El diseño usa los puntos de quiebre de Bootstrap. Cada tarjeta del catálogo declara cuántas columnas ocupa según el ancho: `col-12 col-sm-6 col-md-4 col-lg-3`.

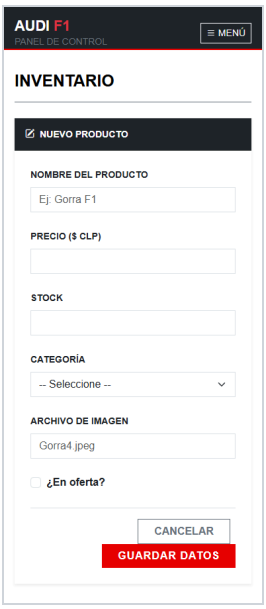
Dispositivo	Ancho	Comportamiento
Celular	< 576 px	1 producto por fila; menú hamburguesa; panel admin con botón "MENÚ"; imágenes del carrito más pequeñas.
Celular grande	≥ 576 px (sm)	2 productos por fila.
Tablet	≥ 768 px (md)	3 productos por fila; el panel admin muestra la barra lateral fija.
Escritorio	≥ 992 px (lg)	4 productos por fila; menú de navegación completo; carrito y resumen en dos columnas.



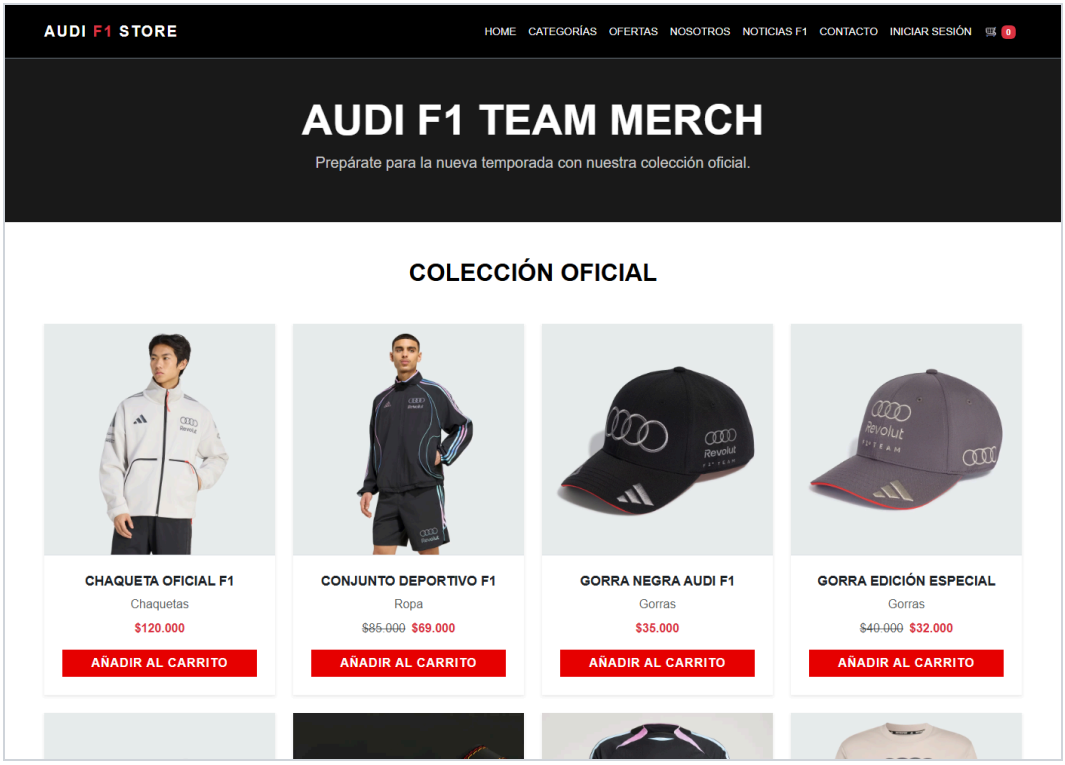
Celular (390 px)



Tablet (820 px)



Panel admin en celular



Escritorio (1400 px): 4 productos por fila y menú completo.

8. Flujo de navegación

Sección	Rutas
Portal del cliente	`/`, `/producto/:id`, `/categorias`, `/ofertas`, `/carrito`, `/checkout`, `/compra-exitosa`, `/compra-fallida`, `/login`, `/registro`, `/nosotros`, `/blog`, `/contacto`
Panel de administración (rutas anidadas)	`/admin`, `/admin/productos`, `/admin/usuarios`
Errores	Cualquier otra ruta muestra la página 404.

Flujo de compra: Catálogo → Carrito (cupón) → Checkout → Compra exitosa (vacía el carrito) o Compra fallida (conserva el carrito para reintentar).

9. Pruebas

El detalle está en el **Documento de Cobertura de Testing**. En resumen: 94 pruebas en 10 archivos, con mocks de `localStorage`, `window.confirm`, del reloj (`jasmine.clock`) y de funciones recibidas por props (`jasmine.createSpy`). La cobertura medida es de 100 % en sentencias, ramas, funciones y líneas.

10. Trazabilidad con la rúbrica

Indicador	Evidencia en el proyecto
IE2.1.1 Framework JS	Proyecto React + Vite estructurado por carpetas; 17 vistas conectadas con React Router (incluida la 404).
IE2.1.2 Props, estado y responsivo	Tabla de componentes (sección 4) y diseño responsivo con Bootstrap (sección 7).
IE2.2.1 Pruebas Jasmine/Karma	`src/tests/` : 94 pruebas que verifican lógica, DOM y eventos.
IE2.3.1 Proceso de testeo	`karma.config.cjs`, mocks y reporte de cobertura en `coverage/index.html`.

11. Conclusión

La versión 2.0 conserva la identidad visual de la Audi F1 Store construida en la EP1 y la lleva a una aplicación React con componentes reutilizables, navegación SPA, diseño responsivo con Bootstrap y un proceso de pruebas unitarias medible y repetible.